

## TESTE DE PERFORMANCE 1



**Engenharia de Software**  
**Projeto de Bloco**  
Aluna: Rafaela Oliveira Ribeiro

**15/05/2026**

## 1. Visão Geral

O tema do sistema escolhido será Flashcards. Em resumo, será um Sistema de Flashcards, aplicação web inspirada no Anki. O usuário cadastra cartões com pergunta e resposta e, no modo de estudo, vê a pergunta, tenta responder mentalmente e depois revela a resposta clicando em um botão.

A inspiração para este sistema foi o convívio com pessoas que diariamente estudam para vestibulares e provas que necessitam de memorização, como em universidades de medicina e direito.

## 2. Arquitetura

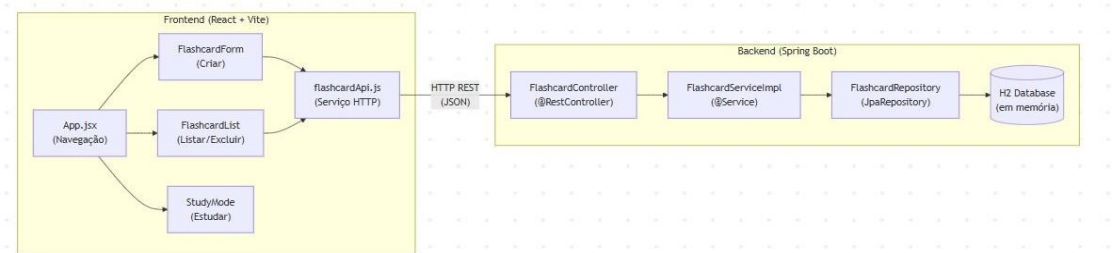
O backend segue o padrão de três camadas, onde cada parte tem uma responsabilidade bem definida:

| Camada                    | Responsabilidade                                                                        |
|---------------------------|-----------------------------------------------------------------------------------------|
| Controller (Apresentação) | Recebe as requisições HTTP, valida os dados de entrada e devolve a resposta ao cliente. |
| Service (Negócio)         | Contém as regras de negócio. Fica entre o Controller e o Repository.                    |
| Repository (Dados)        | Faz o acesso ao banco de dados H2 usando Spring Data JPA.                               |

### Boas práticas aplicadas:

| Padrão              | Como foi aplicado                                                                         |
|---------------------|-------------------------------------------------------------------------------------------|
| SOLID               | Injeção de dependência via construtor e uso de interface no Service.                      |
| DTO Pattern         | FlashcardRequestDTO e FlashcardResponseDTO isolam os dados da API da entidade interna.    |
| Tratamento de erros | GlobalExceptionHandler centraliza os erros retornados pela API com @RestControllerAdvice. |

### Diagrama de Componentes



### 3. Estrutura de Pacotes

#### Backend — pacote raiz: `com.rafaelaribeiro.sistema_flashcards`

| Pacote                   | Descrição                                                                                   |
|--------------------------|---------------------------------------------------------------------------------------------|
| <code>config/</code>     | Configurações de CORS e dados de exemplo carregados na inicialização.                       |
| <code>controller/</code> | <code>FlashcardController</code> — recebe as requisições HTTP.                              |
| <code>dto/</code>        | <code>FlashcardRequestDTO</code> e <code>FlashcardResponseDTO</code> (Java Records).        |
| <code>model/</code>      | <code>Flashcard</code> — entidade JPA que representa a tabela no banco.                     |
| <code>repository/</code> | <code>FlashcardRepository</code> — herda de <code>JpaRepository</code> .                    |
| <code>service/</code>    | Interface <code>FlashcardService</code> e implementação <code>FlashcardServiceImpl</code> . |

#### Frontend — pasta `frontend/src/`

| Arquivo                                   | Descrição                                            |
|-------------------------------------------|------------------------------------------------------|
| <code>App.jsx</code>                      | Raiz da aplicação — controla a navegação entre abas. |
| <code>components/FlashcardForm.jsx</code> | Formulário para criar novos flashcards.              |
| <code>components/FlashcardList.jsx</code> | Lista os flashcards e permite excluir.               |
| <code>components/StudyMode.jsx</code>     | Modo de estudo com flip de cartão.                   |

|                                       |                                            |
|---------------------------------------|--------------------------------------------|
| <code>services/flashcardApi.js</code> | Funções fetch que chamam a API do backend. |
|---------------------------------------|--------------------------------------------|

## 4. API REST

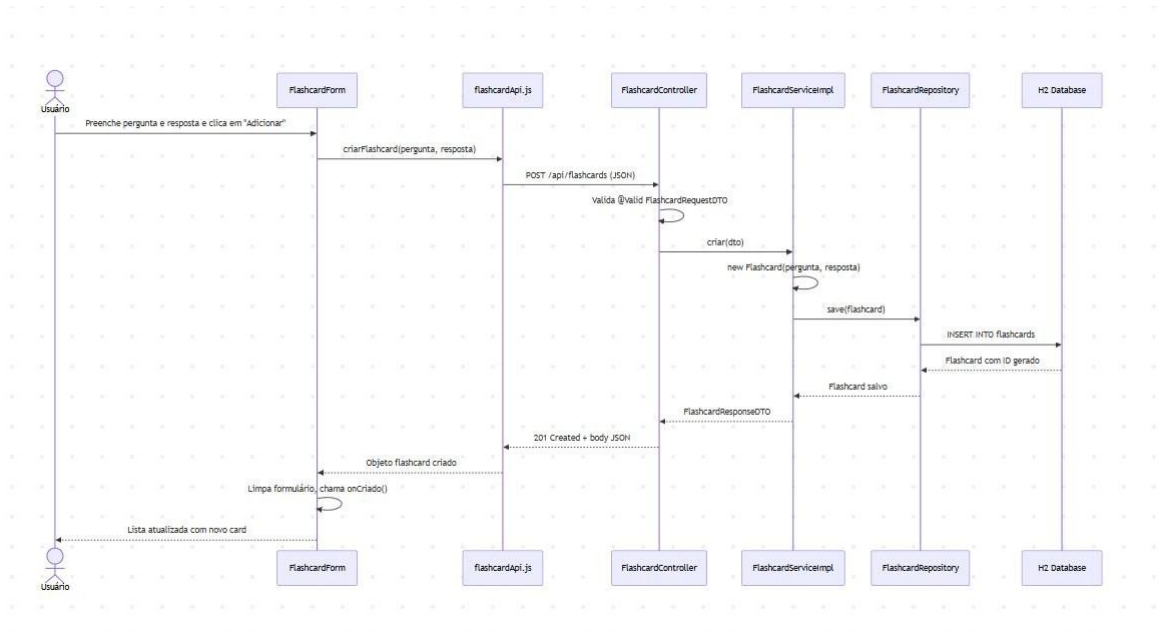
acontece via HTTP com dados em formato JSON. Os endpoints disponíveis são:

| Método | Rota                 | Descrição                                |
|--------|----------------------|------------------------------------------|
| GET    | /api/flashcards      | Retorna todos os flashcards cadastrados. |
| POST   | /api/flashcards      | Cria um novo flashcard.                  |
| DELETE | /api/flashcards/{id} | Remove um flashcard pelo ID.             |

## 5. Diagramas de Sequência

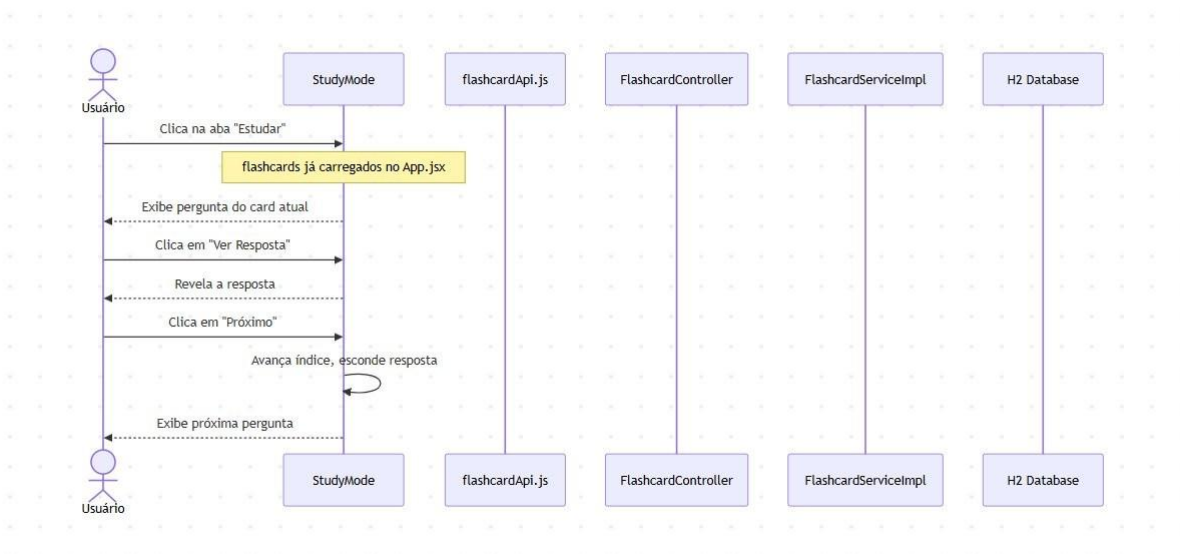
### 5.1 Criação de um Flashcard

O diagrama abaixo mostra o que acontece quando o usuário preenche o formulário e clica em "Adicionar", desde o frontend até o banco de dados:



### 5.2 Modo de Estudo

O modo de estudo usa os flashcards já carregados no App.jsx, sem novas chamadas à API. O usuário navega pelos cartões e revela as respostas localmente:



## 6. Modelagem de Domínio (DDD)

A aplicação foi modelada seguindo conceitos básicos de Domain-Driven Design (DDD), identificando o domínio e a entidade principal do sistema:

|                 |                                    |
|-----------------|------------------------------------|
| Domínio         | Aprendizado por repetição espaçada |
| Subdomínio core | Gestão de Flashcards               |
| Bounded Context | FlashcardContext                   |
| Entidade raiz   | Flashcard                          |

### Campos da entidade Flashcard:

| Campo    | Tipo          | Descrição                                       |
|----------|---------------|-------------------------------------------------|
| id       | Long          | Identificador único gerado automaticamente.     |
| pergunta | String        | Texto da frente do cartão. Campo obrigatório.   |
| resposta | String        | Texto do verso do cartão. Campo obrigatório.    |
| criadoEm | LocalDateTime | Data e hora de criação. Gerado automaticamente. |

Regras: os campos pergunta e resposta não podem ser vazios. O campo criadoEm é preenchido automaticamente no momento da criação.

## 7. Link do Repositório no GitHub

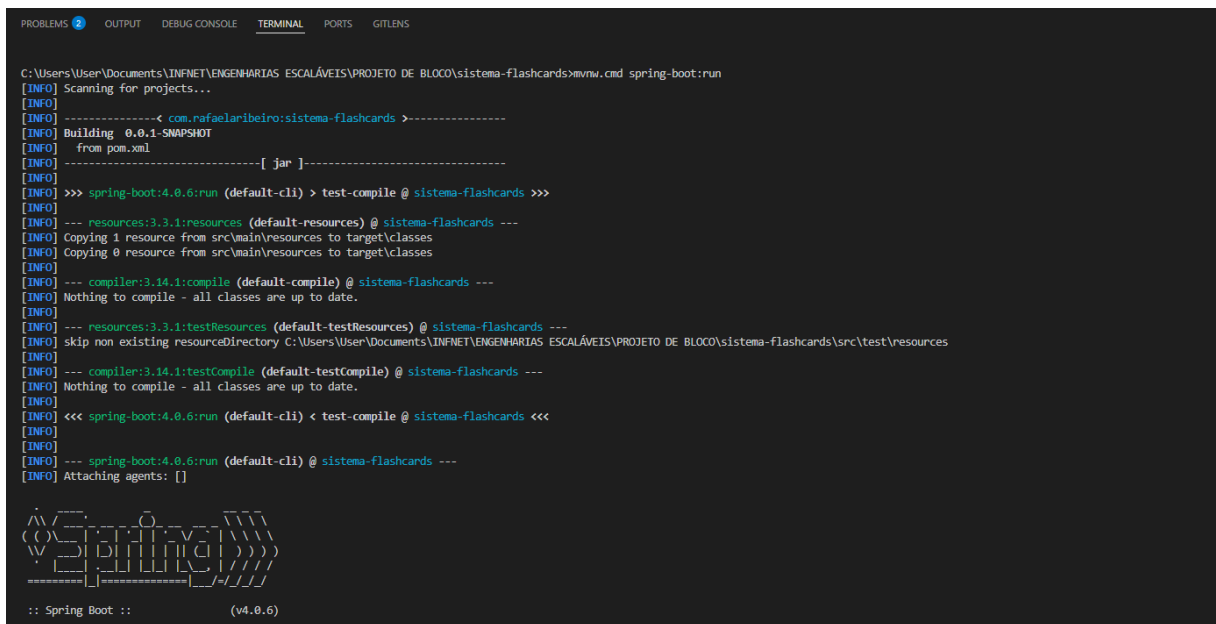
<https://github.com/RafaelaRibe1ro/flashcards-sistema>

## 8. Apresentação do Projeto

A seguir serão apresentados vídeos e imagens explicando como funciona cada parte do sistema e o passo a passo que o usuário precisa realizar para visualizar a aplicação.

## 1. Iniciando o back-end

Abra um terminal na pasta do projeto e execute `mvn spring-boot:run`. O servidor sobe na porta 8080 e cria o banco de dados em memória automaticamente.

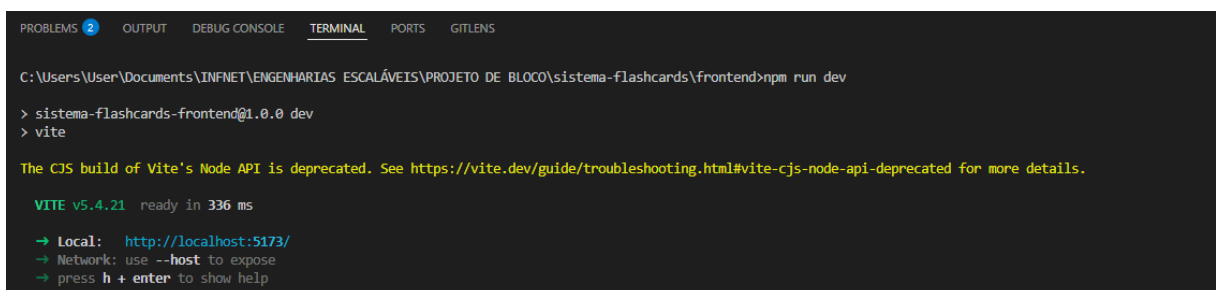


```
C:\Users\User\Documents\INFNET\ENGENHARIAS ESCALÁVEIS\PROJETO DE BLOCO\sistema-flashcards>mvnw.cmd spring-boot:run
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.rafaelaribeiro:sistema-flashcards >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> spring-boot:4.0.6:run (default-cli) > test-compile @ sistema-flashcards >>>
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ sistema-flashcards ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ sistema-flashcards ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ sistema-flashcards ---
[INFO] skip non existing resourceDirectory C:\Users\User\Documents\INFNET\ENGENHARIAS ESCALÁVEIS\PROJETO DE BLOCO\sistema-flashcards\src\test\resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ sistema-flashcards ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] <<< spring-boot:4.0.6:run (default-cli) < test-compile @ sistema-flashcards <<<
[INFO]
[INFO] --- spring-boot:4.0.6:run (default-cli) @ sistema-flashcards ---
[INFO] Attaching agents: []

  ____ _
 / ___ \| | | |
/ /___ \| |_| |
 \___ \|____|_|_|
  Spring Boot (v4.0.6)
```

## 2. Iniciando o front-end

Em outro terminal, entre na pasta frontend e execute `npm run dev`. A interface fica disponível em <http://localhost:5173>.



```
C:\Users\User\Documents\INFNET\ENGENHARIAS ESCALÁVEIS\PROJETO DE BLOCO\sistema-flashcards\frontend>npm run dev
> sistema-flashcards-frontend@1.0.0 dev
> vite

The CJS build of Vite's Node API is deprecated. See https://vite.dev/guide/troubleshooting.html#vite-cjs-node-api-deprecated for more details.

VITE v5.4.21 ready in 336 ms
  → Local: http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

## 3. Visualizando a aplicação (tópicos + vídeo)

- Ao acessar o endereço no navegador, a aplicação abre na aba Flashcards, com o formulário de criação e a lista de cards cadastrados.
- Preencha os campos Pergunta e Resposta e clique em Salvar. O card aparece imediatamente na lista abaixo sem precisar recarregar a página.
- Cada card cadastrado é exibido com sua pergunta e resposta. É possível ver todos os flashcards criados na sessão atual.
- Clique no botão Excluir ao lado do card desejado. O item é removido da lista e do banco de dados instantaneamente.
- Clique na aba Estudar no menu superior. O sistema exibe o primeiro flashcard mostrando apenas a pergunta.
- Clique no botão Revelar resposta. A resposta aparece abaixo da pergunta no mesmo card.
- Use os botões Anterior e Próximo para navegar pelos flashcards cadastrados. O contador no topo mostra a posição atual.

Para visualizar o vídeo, acesse:

<https://github.com/RafaelaRibeiro/flashcards-sistema/blob/master/docs/v%C3%ADdeo/Grava%C3%A7%C3%A3o%20de%20Tela%202026-05-15%20135856.mp4>

ou também:

<https://drive.google.com/file/d/11VHi88gs688SUI6Rje7rNlvXzGNaSPGS/view?usp=sharing>